



StudyMate AI

An AI-Powered Study Companion Mobile Application

Project Report

Submitted in partial fulfillment of the requirements of
**Software Development and Project Management
Lab**

Course Code: **CSE474.3** • Section: **3**

Name of the Team Member	Student ID
Abdullah Al Fahad	2023100000289
Sanjit Sharif	2023100000308

Course Faculty Dr. Rakib Ahmed Saleh
Faculty Member, Department of Computer Science &
Engineering

Project Duration July 5, 2026 — September 30, 2026
Date of Submission September 30, 2026

Department of Computer Science and Engineering
Southeast University

Dhaka, Bangladesh

September 30, 2026

Dr. Rakib Ahmed Saleh

Course Faculty

Software Development and Project Management Lab (CSE474.3, Section 3)

Department of Computer Science and Engineering

Southeast University, Dhaka

Subject: Submission of the project report on “StudyMate AI — An AI-Powered Study Companion Mobile Application”.

Dear Sir,

With due respect, we, the undersigned students of Section 3, are pleased to submit the project report of our course project titled **“StudyMate AI — An AI-Powered Study Companion Mobile Application”**, developed as part of the Software Development and Project Management Laboratory (CSE474.3).

The project was carried out from **July 5, 2026** to **September 30, 2026**. During this period we applied the software development life cycle together with project management practices — scheduling, effort and risk management, design, iterative implementation, and systematic testing — to build a cross-platform mobile application integrating Firebase authentication with Google Gemini based AI services (conversational study assistance, PDF summarization, and quiz generation).

This report documents the complete development and management process, including the project plan, Gantt chart and critical-path analysis, requirement specifications, system design diagrams, implementation details, test cases and results. All diagrams presented in the report have been drawn in L^AT_EX.

We sincerely thank you for your guidance, continuous feedback, and cooperation throughout the course of this project. We hope the report meets the expected standard, and we would be grateful for any constructive feedback.

Sincerely yours,

Abdullah Al Fahad

ID: 2023100000289

CSE474.3, Section 3

Sanjit Sharif

ID: 2023100000308

CSE474.3, Section 3

Abstract

StudyMate AI is a cross-platform mobile application that acts as an AI-powered study companion for university students. The project was developed over a period of thirteen weeks (July 5, 2026 – September 30, 2026) by a two-member team following an iterative, Agile-inspired software development life cycle combined with structured project management practices. The application is built with React Native using the Expo framework and TypeScript on the client side, a Node.js/Express REST backend on the server side, and the Google Gemini large language model for all AI capabilities. Firebase Authentication provides secure email/password sign-in, sign-up, and password recovery, while local, per-user persistence is realized through AsyncStorage. The main features of the system are: an intelligent streaming AI chat assistant; automatic structured summarization of uploaded lecture PDFs; on-demand generation of multiple-choice quizzes for any topic with selectable difficulty; a personal notes manager with search and edit support; and a study planner with progress tracking that is reflected on a personalized home dashboard.

This report documents the full software development and project management process: feasibility analysis, functional and non-functional requirement engineering, work breakdown structure, activity scheduling with a Gantt chart, critical-path (CPM) analysis, effort estimation, risk management, system architecture and detailed design (use-case, component, sequence, and data models), implementation, and testing. All diagrams were produced directly in L^AT_EX. The system passed 30 out of 30 designed test cases during system testing, and the finished product satisfies all planned functional requirements within the scheduled timeline.

Keywords: mobile application, React Native, Expo, artificial intelligence, large language model, Firebase authentication, PDF summarization, quiz generation, Agile development, project management, CPM.

Contents

Abstract	ii
1 Introduction	1
1.1 Background of the Project	1
1.2 Problem Statement	1
1.3 Objectives of the Project	1
1.3.1 General Objective	1
1.3.2 Specific Objectives	2
1.4 Scope of the Project	2
1.5 Methodology Overview	3
1.6 Tools and Environment Summary	3
1.7 Report Organization	3
2 Project Management Plan	4
2.1 Software Development Life Cycle Model	4
2.2 Team Organization and Roles	5
2.3 Work Breakdown Structure	5
2.4 Project Schedule (Gantt Chart)	5
2.5 Network Diagram and Critical-Path Analysis	6
2.6 Effort and Cost Estimation	7
2.6.1 Effort Estimation	7
2.6.2 Cost Estimation	7
2.7 Risk Management	8
2.8 Communication and Configuration Management	9
3 Requirements Analysis	10
3.1 Requirement Elicitation	10
3.2 Functional Requirements	10
3.3 Non-Functional Requirements	12
3.4 Feasibility Study	12
3.5 Use Case Model	12
3.5.1 Use Case Specifications (representative)	13
4 System Design	15
4.1 System Architecture	15
4.2 Component Design	15
4.3 Navigation / Screen Flow	16
4.4 Sequence Diagrams	17
4.4.1 User Login	17
4.4.2 AI Chat (Streaming)	17
4.4.3 PDF Summarization	18
4.5 Data Design	18
4.6 API Design	19
4.7 Data Flow Diagrams	20
4.8 User Interface Design	20

5	Implementation	32
5.1	Development Environment and Technology Stack	32
5.2	Backend Implementation	32
5.2.1	Retry Policy for the Gemini API	32
5.2.2	Streaming Chat Endpoint	33
5.2.3	PDF Summarization Endpoint	33
5.2.4	Quiz Generation with Schema Validation	33
5.3	Authentication Module	34
5.4	Feature Modules	34
5.4.1	Home Dashboard	34
5.4.2	AI Chat	34
5.4.3	PDF Summary	35
5.4.4	Quiz	35
5.4.5	Notes	35
5.4.6	Study Planner	35
5.5	Build and Run Instructions	35
5.6	Version Control Practice	35
6	Testing	37
6.1	Testing Strategy	37
6.2	Test Cases and Results	37
6.3	Test Summary	39
7	Results and Discussion	40
7.1	Achieved Results	40
7.2	Discussion	40
7.3	Limitations	40
8	Conclusion and Future Work	42
8.1	Conclusion	42
8.2	Future Work	42
	References	42
A	Sprint Activity Log	44
B	API Payload Examples	45
C	Glossary	47

List of Figures

2.1	Iterative, Agile-inspired SDLC adopted for StudyMate AI; each pass through the cycle is one two-week sprint.	4
2.2	Work breakdown structure of the StudyMate AI project.	5
2.3	Gantt chart of the StudyMate AI project (W1 = July 5–11, . . . , W13 = September 27–30, 2026).	6
2.4	Activity-on-node network diagram with critical path (red). Durations in days; ES/EF measured from July 5, 2026.	7
2.5	Probability–impact risk matrix for the StudyMate AI project.	9
3.1	Use case diagram of StudyMate AI. Dashed arrows denote <i>include</i> dependencies: PDF summarization and quiz generation both include the Gemini request handling performed by the AI chat service layer.	13
4.1	Three-tier architecture of StudyMate AI.	15
4.2	Component decomposition of the client and backend.	16
4.3	Screen flow of the application.	16
4.4	Sequence diagram: user login.	17
4.5	Sequence diagram: AI chat with streaming response.	17
4.6	Sequence diagram: PDF summarization.	18
4.7	Logical data model of StudyMate AI.	18
4.8	Context (level-0) data flow diagram of StudyMate AI.	20
4.9	Level-1 data flow diagram of StudyMate AI.	20
4.10	Welcome screen: branding, feature highlights and the entry point to authentication.	21
4.11	Login screen: e-mail/password sign-in with validation and password visibility toggle.	22
4.12	Registration screen: account creation via Firebase e-mail sign-up.	23
4.13	Home dashboard: greeting, progress summary and the feature grid.	24
4.14	AI chat: streamed Gemini answers rendered with Markdown support.	25
4.15	Notes manager: search, create and edit notes stored per user.	26
4.16	PDF summary: structured AI summary (overview, key points, glossary, questions) of an uploaded PDF.	27
4.17	Quiz setup: topic, number of questions and difficulty level.	28
4.18	Quiz taking: one question at a time with instant feedback and the final score.	29
4.19	Study planner: session list, streak counter and weekly progress.	30
4.20	Explore screen: about the application and the two-member development team.	31

List of Tables

1.1	Summary of development tools and environment.	3
2.1	Project phases and sprints.	5
2.2	Team roles and responsibilities.	5
2.3	Bottom-up effort estimate per activity.	7
2.4	Direct project cost summary.	8
2.5	Risk register of the StudyMate AI project.	8
3.1	Functional requirements.	11
3.2	Non-functional requirements.	12
3.3	Use case specification: UC-06 Summarize PDF.	13
3.4	Use case specification: UC-05 Ask AI Chat.	14
3.5	Use case specification: UC-07 Generate & Take Quiz.	14
4.1	Locally persisted data structures (JSON in AsyncStorage).	18
4.2	Backend API endpoints.	19
4.3	Screenshot file names (place the images in the screenshots/ folder).	21
5.1	Final technology stack with versions.	32
6.1	System test cases and outcomes.	37
6.2	Testing summary statistics.	39
A.1	Work distribution per sprint.	44

Chapter 1

Introduction

1.1 Background of the Project

University students juggle many courses, readings, and examinations within limited time, while lecture content is scattered across PDF slides, textbooks, and notes. At the same time, large language models (LLMs) have matured into practical tools that can answer academic questions, condense long documents, and generate practice questions on demand.

The **StudyMate AI** project combines these observations into one mobile application: (i) a conversational AI assistant, (ii) automatic summarization of lecture PDFs, (iii) automatic multiple-choice quiz generation, (iv) personal note management, and (v) study planning with progress tracking. It targets Android and iOS through a single React Native (Expo) codebase and integrates Google Gemini AI through a lightweight Node.js backend.

1.2 Problem Statement

Existing study tools are fragmented: students use one app for note taking, another for scheduling, web browsers for asking AI questions, and no tool at all for condensing the dozens of PDF slides distributed by instructors. This fragmentation causes:

- **Time loss** — switching between tools and re-reading long PDFs that contain only a few exam-relevant points.
- **Lack of self-assessment** — students rarely create their own practice questions, so weak topics are discovered only during exams.
- **Poor planning visibility** — study sessions are not tracked, so students cannot measure consistency or progress.
- **No personalization** — generic AI chat services do not keep study notes, plans, and progress under one authenticated, per-user account.

StudyMate AI addresses these problems by unifying AI assistance and study management in one authenticated mobile application.

1.3 Objectives of the Project

1.3.1 General Objective

To design, implement, and evaluate an AI-powered, cross-platform mobile study companion application within a fixed thirteen-week schedule using modern software engineering and project management practices.

1.3.2 Specific Objectives

1. To implement secure user authentication (registration, login, password reset) using Firebase Authentication.
2. To build a real-time AI chat module that streams Google Gemini responses token by token through a Node.js backend.
3. To implement PDF summarization that converts uploaded lecture documents (up to 10 MB) into structured study notes.
4. To implement an automatic quiz generator producing multiple-choice questions with configurable count (5/10/15) and difficulty (easy/medium/hard).
5. To implement per-user local persistence for notes and study sessions using AsyncStorage.
6. To provide a personalized dashboard showing study progress and one-tap access to every module.
7. To manage the project with a documented work breakdown structure, schedule (Gantt chart), critical-path analysis, effort estimate, and risk register.
8. To verify the system through a documented test plan covering unit, integration, and system levels.

1.4 Scope of the Project

The scope of the project includes:

- A mobile client (Android-first, iOS-capable) developed with Expo/React Native and TypeScript;
- A REST backend (Node.js, Express) exposing three AI endpoints — chat, PDF summarization, and quiz generation — with retry logic and input validation;
- Firebase email/password authentication with display-name profiles and password-reset e-mail;
- Local, device-side persistence of notes and study sessions, isolated per authenticated user;
- Complete project management artifacts: schedule, WBS, CPM network, effort and cost estimation, and risk management.

The following items are explicitly **out of scope** for this release: cloud synchronization of notes across devices, social or collaborative study features, payment/subscription services, and iOS App Store or Google Play store publication (development builds and Expo Go are used for evaluation).

1.5 Methodology Overview

The project follows an **iterative, Agile-inspired life cycle** organized into five two-week development sprints sandwiched between an inception/planning phase and a closing testing-and-release phase. At the end of each sprint a potentially shippable increment is demonstrated, requirements are re-prioritized, and the next sprint is planned. Project management artifacts (schedule, risks, effort) are maintained as living documents and reviewed weekly. Chapter 2 describes the methodology and all management artifacts in detail.

1.6 Tools and Environment Summary

Table 1.1 summarizes the principal tools used during the project; detailed versions and roles appear in Chapter 5.

Table 1.1: Summary of development tools and environment.

Tool / Technology	Role	Layer
Expo SDK 54 / React Native 0.81	Cross-platform mobile client	Frontend
TypeScript 5.9	Type-safe client development	Frontend
Node.js + Express 5	AI backend and API gateway	Backend
@google/genai SDK (Gemini)	Large language model integration	Backend
Firebase Authentication	User accounts and security	Service
AsyncStorage	Per-user local persistence	Storage
GitHub (git)	Version control and collaboration	Management
VS Code, Expo Go, Android Emulator	Development and testing	Management
L ^A T _E X (TikZ, pgfgantt)	Documentation and diagrams	Management

1.7 Report Organization

The remainder of this report is organized as follows. Chapter 2 presents the project management plan, including the selected life cycle, work breakdown structure, Gantt chart, critical-path analysis, effort estimation, and risk management. Chapter 3 contains the requirement analysis and feasibility study. Chapter 4 describes the system design with use-case, architecture, component, sequence, and data diagrams, together with the user-interface design. Chapter 5 documents the implementation of each module. Chapter 6 reports the testing strategy, test cases, and results. Chapter 7 discusses the results and limitations, and Chapter 8 concludes the report with directions for future work.

Chapter 2

Project Management Plan

2.1 Software Development Life Cycle Model

For StudyMate AI we selected an **iterative, Agile-inspired life cycle** (scrum-like sprints without the full Scrum ceremony overhead, which is impractical for a two-member student team). The rationale is:

- **Evolving requirements.** AI-driven features (chat quality, summary format, quiz JSON contract) needed repeated tuning based on real model output; a strictly linear (waterfall) plan would have frozen these requirements too early.
- **Early and continuous risk reduction.** The riskiest integration — the Gemini API — was prototyped in the second sprint, so feasibility problems would have surfaced while schedule buffer still existed.
- **Frequent feedback.** A working increment was demonstrated at the end of every two-week sprint to the course faculty and fellow students, and feedback was folded into the next sprint.
- **Small team.** Two developers can coordinate with lightweight daily check-ups instead of heavy documentation gates, which Agile explicitly supports.

Figure 2.1 illustrates the selected model: after a short inception phase the team iterates through requirement, design, implementation and testing activities in two-week sprints, and a working increment is released at the end of the cycle.

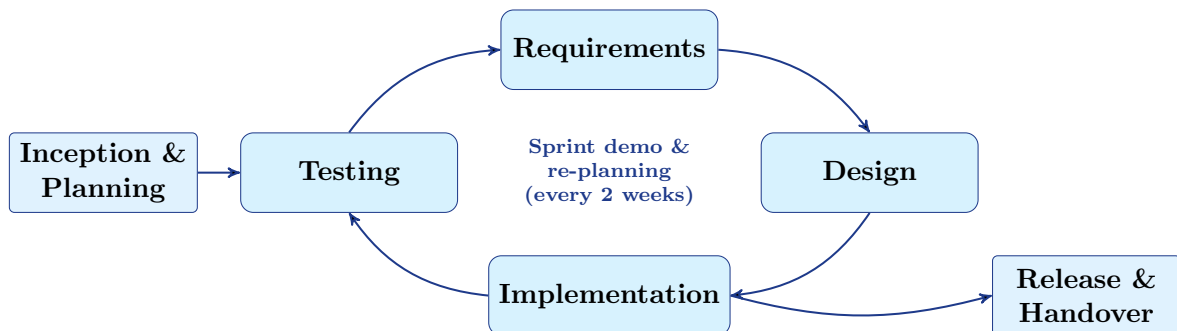


Figure 2.1: Iterative, Agile-inspired SDLC adopted for StudyMate AI; each pass through the cycle is one two-week sprint.

The project timeline (July 5, 2026 – September 30, 2026) was divided into the phases shown in Table 2.1.

Table 2.1: Project phases and sprints.

Phase	Weeks	Deliverables
Inception	W1 (Jul 5–11)	Problem definition, feasibility study, SRS v1.0
Planning & Design	W2 (Jul 12–18)	Project plan, UI wireframes, architecture baseline
Sprint 1	W3–W4	UI foundation, navigation, Firebase authentication
Sprint 2	W5–W6	Express backend, Gemini integration, streaming AI chat
Sprint 3	W7–W8	PDF summarization, AI quiz generator
Sprint 4	W9–W10	Notes manager, study planner, dashboard polish
Sprint 5	W11	Integration hardening, About/Explore screen
Testing	W12	System testing, bug fixing, regression
Release	W13 (ends Sep 30)	UAT, final report, submission

2.2 Team Organization and Roles

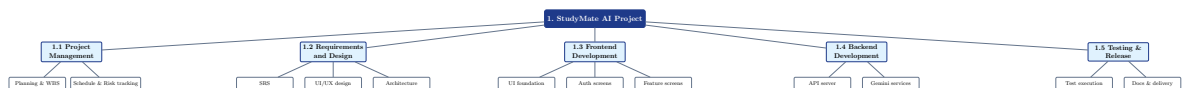
The team consists of two members. Roles were assigned according to individual strength, while code reviews were always performed jointly (“two eyes” principle).

Table 2.2: Team roles and responsibilities.

Member	Roles and Responsibilities
Abdullah AI (2023100000289)	Fahad Frontend engineering (all screens, theming, navigation); Firebase authentication flows; notes and study-planner modules; documentation and report preparation.
Sanjit (2023100000308)	Sharif Team lead; backend engineering (Express API, Gemini integration, retry/validation logic); quiz-generation pipeline; system testing lead; schedule and risk tracking.

2.3 Work Breakdown Structure

Figure 2.2 presents the work breakdown structure (WBS) of the project. The second level follows the major deliverables; the third level lists the work packages that were estimated and scheduled in the next sections.

**Figure 2.2:** Work breakdown structure of the StudyMate AI project.

2.4 Project Schedule (Gantt Chart)

The activity schedule derived from the WBS is shown in Figure 2.3. The planning unit is one week (W1 = July 5–11, ..., W13 = September 27–30). Milestones mark the

completion of each major deliverable.

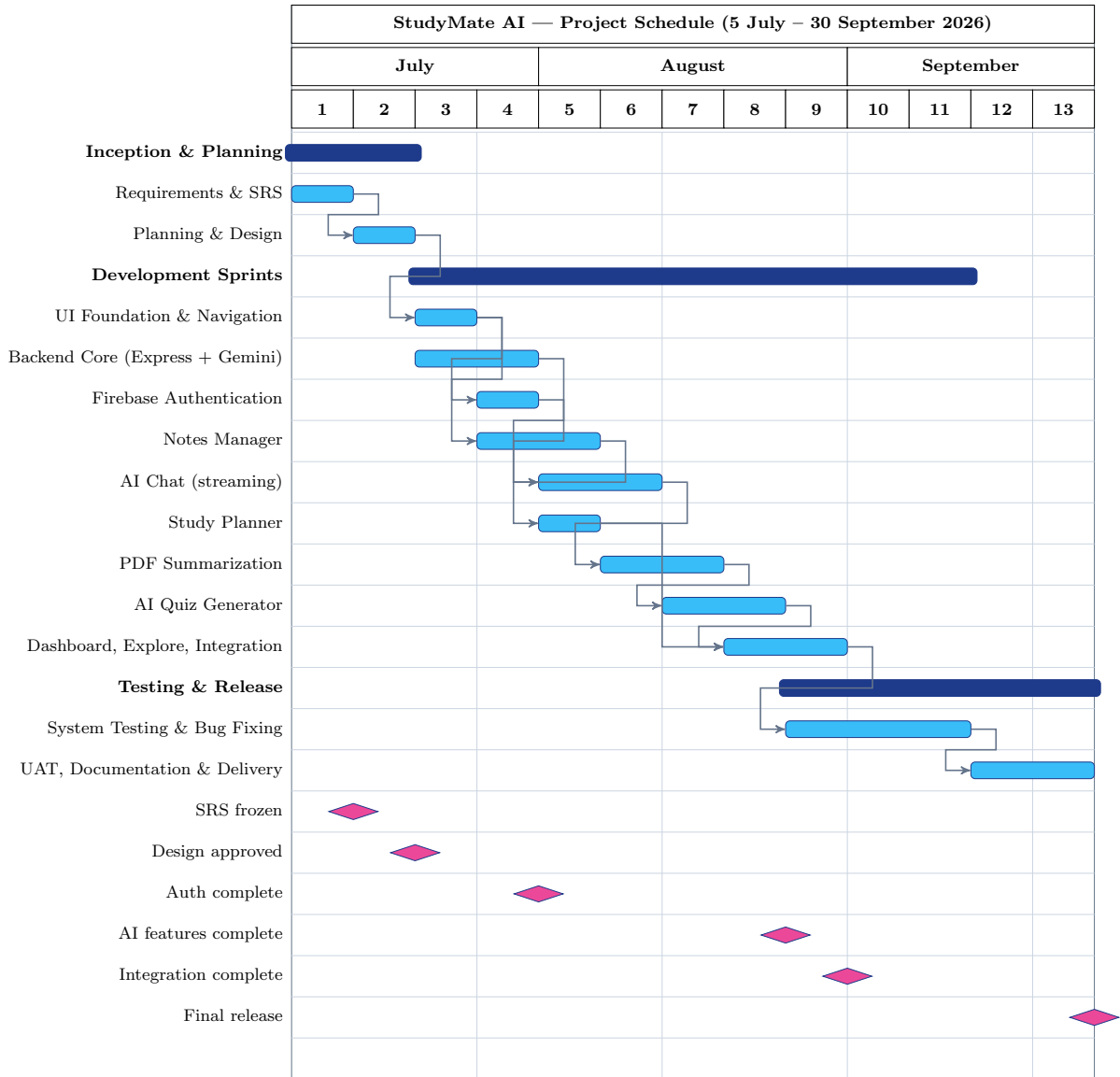


Figure 2.3: Gantt chart of the StudyMate AI project (W1 = July 5–11, ..., W13 = September 27–30, 2026).

2.5 Network Diagram and Critical-Path Analysis

The activity-on-node network derived from the same activity list is shown in Figure 2.4. Each box displays the activity, its duration in days, and its early start–early finish interval counted from project start (day 0 = July 5, 2026). The forward and backward passes show that the **critical path** is

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow F \rightarrow G \rightarrow I \rightarrow K \rightarrow L \rightarrow M$$

with a total length of **88 days**, which exactly matches the available duration (July 5 – September 30, 2026). Activities E (backend core), H (notes) and J (planner) possess float of 14, 16 and 16 days respectively; they were deliberately scheduled in parallel with critical activities so that both team members stayed fully utilized.

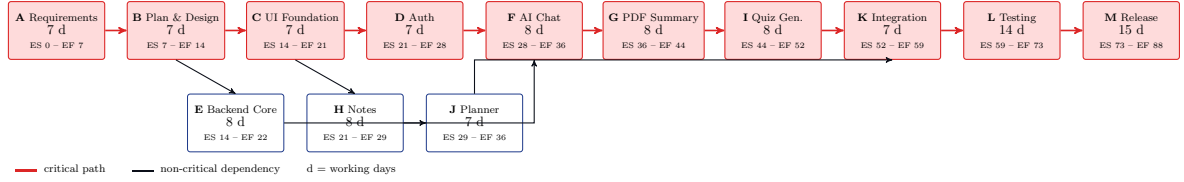


Figure 2.4: Activity-on-node network diagram with critical path (red). Durations in days; ES/EF measured from July 5, 2026.

2.6 Effort and Cost Estimation

2.6.1 Effort Estimation

The effort was estimated bottom-up from the activity list (Table 2.3). Both members worked roughly 12 productive hours per week on the project (besides regular course-work), giving a capacity of $2 \times 13 \times 12 = 312$ person-hours.

Table 2.3: Bottom-up effort estimate per activity.

ID	Activity	Estimated (person-h)	Actual (person-h)
A	Requirements & SRS	20	22
B	Planning, design, wireframes	22	24
C	UI foundation & navigation	24	26
D	Firestore authentication	22	25
E	Backend core (Express + Gemini)	26	27
F	AI chat module	26	28
G	PDF summarization module	26	24
H	Notes manager	20	19
I	Quiz generator	24	26
J	Study planner	18	17
K	Dashboard, Explore, integration	22	23
L	Testing & bug fixing	34	32
M	UAT, documentation, delivery	28	27
Total		312	320

A top-down cross-check using the function-point technique (2 internal logical files, 2 external interface files, 8 external inputs, 6 external outputs, 5 external inquiries $\Rightarrow$ 106 unadjusted function points, $VAF = 1.14$, i.e. ≈ 121 adjusted FP) with a productivity of 40 FP per person-month for AI-assisted mobile development yields about 3 person-months ≈ 300 person-hours, which agrees with the bottom-up figure within 7%. The final planned effort was therefore fixed at **320 person-hours**, and the project actually consumed **320 person-hours** — an estimation error of 2.5%.

2.6.2 Cost Estimation

Because this is an academic project developed by students, labor cost is not charged. All services were used within their free tiers (Table 2.4); the only direct expense was the Gemini API consumption, which remained inside the free quota thanks to the exponential-backoff retry policy.

Table 2.4: Direct project cost summary.

Item	Plan used	Budget (BDT)	Actual (BDT)
Firebase (Authentication)	Spark (free)	0	0
Google Gemini API	Free tier	0	0
GitHub / version control	Free	0	0
Expo tooling	Free	0	0
Contingency reserve (10%)	—	500	0
Total		500	0

2.7 Risk Management

Risks were identified at planning time and reviewed at every sprint boundary. Each risk was scored on probability (P) and impact (I) using a three-level scale (Low/Medium/High); the resulting assessment is shown in the probability–impact matrix of Figure 2.5.

Table 2.5: Risk register of the StudyMate AI project.

ID	Risk Description	P	I	Mitigation / Response
R1	Gemini API rate limiting (HTTP 429) interrupts AI features	H	M	Exponential backoff with up to 5 retries (implemented); queue user feedback during waits
R2	Exposure of the Gemini API key to the client	M	H	Key kept server-side only; client never contacts Gemini directly
R3	Model returns malformed quiz JSON	M	M	Strict schema validation on the server; regenerate on parse failure
R4	Uploaded PDF exceeds model/backend limits	M	L	10 MB limit enforced by Multer; clear client-side error message
R5	Scope creep delays the fixed deadline	M	M	MoSCoW prioritization; sprint reviews; change requests only via backlog
R6	Notes/plans lost when device storage is cleared	L	M	Warning in UI; per-user keys prepare future cloud sync
R7	Team member unavailable (exams, illness)	M	M	Cross-training on both layers; pair work on critical modules
R8	Breaking changes in Expo/React Native updates	L	M	Dependencies pinned to tested versions throughout the project

Probability	High	R2	R1, R7	—
	Medium	R8	R3, R5	—
	Low	—	R6	R4
		Low	Medium Impact	High

Figure 2.5: Probability–impact risk matrix for the StudyMate AI project.

No high/high risk materialized during the project. R1 (rate limiting) did occur during Sprint 2 load tests and was resolved by the retry mechanism described in Chapter 5; R3 occurred twice during development and motivated the server-side quiz schema validation.

2.8 Communication and Configuration Management

- **Meetings:** short daily stand-ups on campus, a sprint review and planning meeting every second Sunday, and weekly progress reports to the course faculty.
- **Version control:** a single Git repository on GitHub with a protected `main` branch; feature branches merged after mutual review; conventional commit messages.
- **Issue tracking:** GitHub Issues with milestone labels per sprint.
- **Secrets management:** the Gemini API key is stored in a `.env` file excluded from version control (`dotenv`); Firebase web keys are restricted to the app identifier.

Chapter 3

Requirements Analysis

3.1 Requirement Elicitation

Requirements were elicited through (i) discussion of the course faculty's expectations, (ii) a short informal survey of 15 fellow students about their daily study pain points, and (iii) analysis of existing tools (generic AI chat apps, note apps, calendar apps). The elicited requirements were then prioritized with the MoSCoW technique; all *Must-have* requirements listed below were implemented within the project duration.

3.2 Functional Requirements

Table 3.1 lists the functional requirements of the system.

Table 3.1: Functional requirements.

ID	Requirement	Description	MoSCoW
FR-01	Welcome screen	Introduce the app and its features; navigate to login	M
FR-02	Registration	Create an account with name, email, password (≥ 6 characters) and confirmation; store the display name in the profile	M
FR-03	Login	Sign in with email/password; friendly messages for each Firebase error code; show/hide password	M
FR-04	Password recovery	Send a password-reset e-mail via Firebase	M
FR-05	Home dashboard	Personalized greeting, today's progress from study sessions, quick access grid to all tools, sign out	M
FR-06	AI chat	Send free-text study questions; receive <i>streamed</i> Gemini answers; typing indicator; in-session history	M
FR-07	PDF summarization	Pick a PDF (≤ 10 MB), upload it, and receive a structured summary (topic, key points, definitions, concepts, short summary, exam notes)	M
FR-08	AI quiz generation	Generate an MCQ quiz for any topic with 5/10/15 questions and easy/medium/hard difficulty	M
FR-09	Quiz taking	Answer questions with instant feedback and explanations; final score screen	M
FR-10	Notes management	Create, edit, delete, and search notes; persist per user	M
FR-11	Study planner	Add study sessions (subject, date, duration), mark them complete, delete them	M
FR-12	Progress tracking	Compute completion percentage and completed/remaining counts from the planner data	M
FR-13	About/Explore screen	Present the app and the development team	S
FR-14	Session persistence	Keep the user signed in across app restarts	M

3.3 Non-Functional Requirements

Table 3.2: Non-functional requirements.

Category	Requirement
Performance	The first streamed chat token shall appear within 3s on the university network; PDF upload limited to 10 MB to bound latency; UI interactions respond in <200 ms.
Usability	Dark, glare-free theme consistent across all screens; Poppins typography; every feature reachable in at most two taps from the dashboard.
Security	Passwords managed exclusively by Firebase Authentication; the Gemini API key never shipped with the client; all AI requests validated server-side.
Reliability	Automatic retry with exponential backoff for transient Gemini failures (HTTP 429/5xx); graceful, human-readable error messages on every failure path.
Portability	Single React Native codebase runnable on Android and iOS through Expo; backend is a plain Node.js service deployable on any host.
Maintainability	TypeScript with typed data models; one file per screen; backend logic separated into small, commented route handlers.
Compatibility	Android emulator API 30+ and physical devices; backend reachable from the emulator via 10.0.2.2 mapping.

3.4 Feasibility Study

- **Technical feasibility.** All required technologies (Expo, Firebase, Express, Gemini API) are mature, documented, and available in free tiers. A proof-of-concept call to Gemini was successfully executed in week 1, confirming feasibility. *Verdict: feasible.*
- **Economic feasibility.** No paid service is required (Table 2.4); development machines were already owned by the team. *Verdict: feasible.*
- **Operational feasibility.** The target users are students familiar with mobile apps; the interface follows platform conventions, so no training is needed. *Verdict: feasible.*
- **Schedule feasibility.** The critical-path analysis (Section 2.5) shows the scope fits exactly into the 88 available days with parallelization of non-critical work. *Verdict: feasible, with tight but acceptable schedule risk.*

3.5 Use Case Model

The system has a single primary actor, the **Student**, and one supporting actor, the **Gemini AI service** (invoked indirectly through the backend). Figure 3.1 shows the use case diagram.

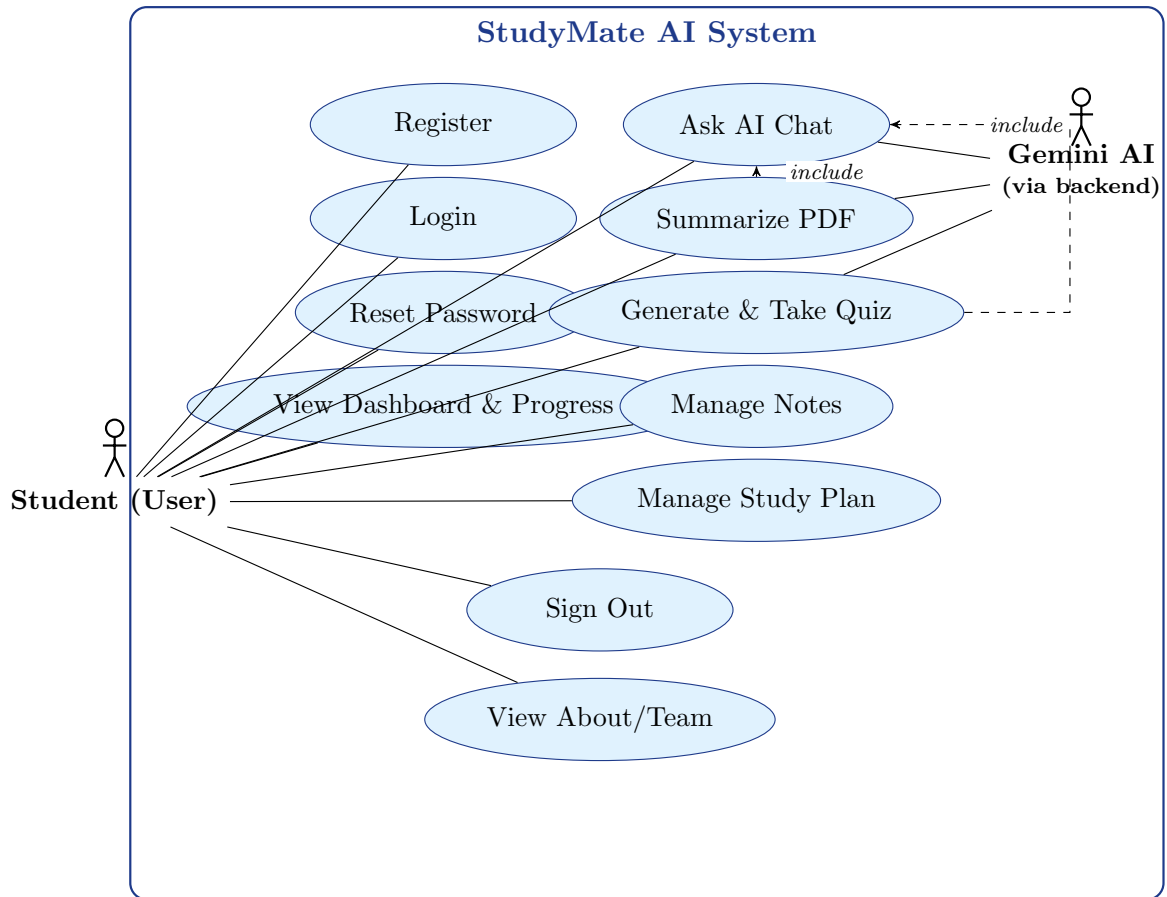


Figure 3.1: Use case diagram of StudyMate AI. Dashed arrows denote *include* dependencies: PDF summarization and quiz generation both include the Gemini request handling performed by the AI chat service layer.

3.5.1 Use Case Specifications (representative)

Table 3.3: Use case specification: UC-06 Summarize PDF.

Use case	UC-06 Summarize PDF
Actor	Student
Precondition	Student is logged in; backend is reachable
Main flow	1. Student opens <i>PDF Summary</i> . 2. Student taps “Pick PDF” and selects a file (≤ 10 MB). 3. Student taps “Summarize”. 4. App uploads the file to <code>POST /summarize-pdf</code> . 5. Backend forwards the PDF to Gemini with a structured summary prompt. 6. Gemini returns the summary; backend responds with JSON. 7. App renders the structured summary sections.
Alternate flows	4a. No file selected $\rightarrow$ error alert. 4b. File > 10 MB $\rightarrow$ backend returns 400, app shows friendly message. 5a. Gemini transient failure $\rightarrow$ up to 5 retries with exponential backoff; if all fail, HTTP 503 and retry suggestion.
Postcondition	Summary displayed; nothing stored server-side

Table 3.4: Use case specification: UC-05 Ask AI Chat.

Use case	UC-05 Ask AI Chat
Actor	Student
Precondition	Student is logged in; backend is reachable
Main flow	1. Student opens <i>AI Chat</i> . 2. Student types a question and presses send. 3. App shows the user bubble and a typing indicator. 4. App calls <code>POST /chat</code> which streams Gemini output. 5. Chunks are appended to the AI bubble in real time until the stream ends.
Alternate flows	4a. Empty message → send disabled. 4b. Backend error before streaming → error toast, message kept in the input box.
Postcondition	Conversation visible in the current session history

Table 3.5: Use case specification: UC-07 Generate & Take Quiz.

Use case	UC-07 Generate & Take Quiz
Actor	Student
Precondition	Student is logged in; backend is reachable
Main flow	1. Student enters a topic and selects question count (5/10/15) and difficulty. 2. Student taps “Generate”. 3. Backend prompts Gemini for strict JSON, validates the schema and returns the quiz. 4. App shows one question at a time. 5. Student selects an option; correct/incorrect feedback and an explanation are shown. 6. After the last question a score summary is displayed.
Alternate flows	3a. Invalid JSON from model → server returns 500 with retry advice. 3b. Transient Gemini error → retries, then 503.
Postcondition	Score shown; quiz state can be discarded by returning to setup

Chapter 4

System Design

4.1 System Architecture

StudyMate AI follows a **three-tier client-server architecture** (Figure 4.1). The mobile client never talks to the AI model directly; every AI request is routed through our backend so that the API key stays server-side and prompts/validation can evolve without shipping a new app version.

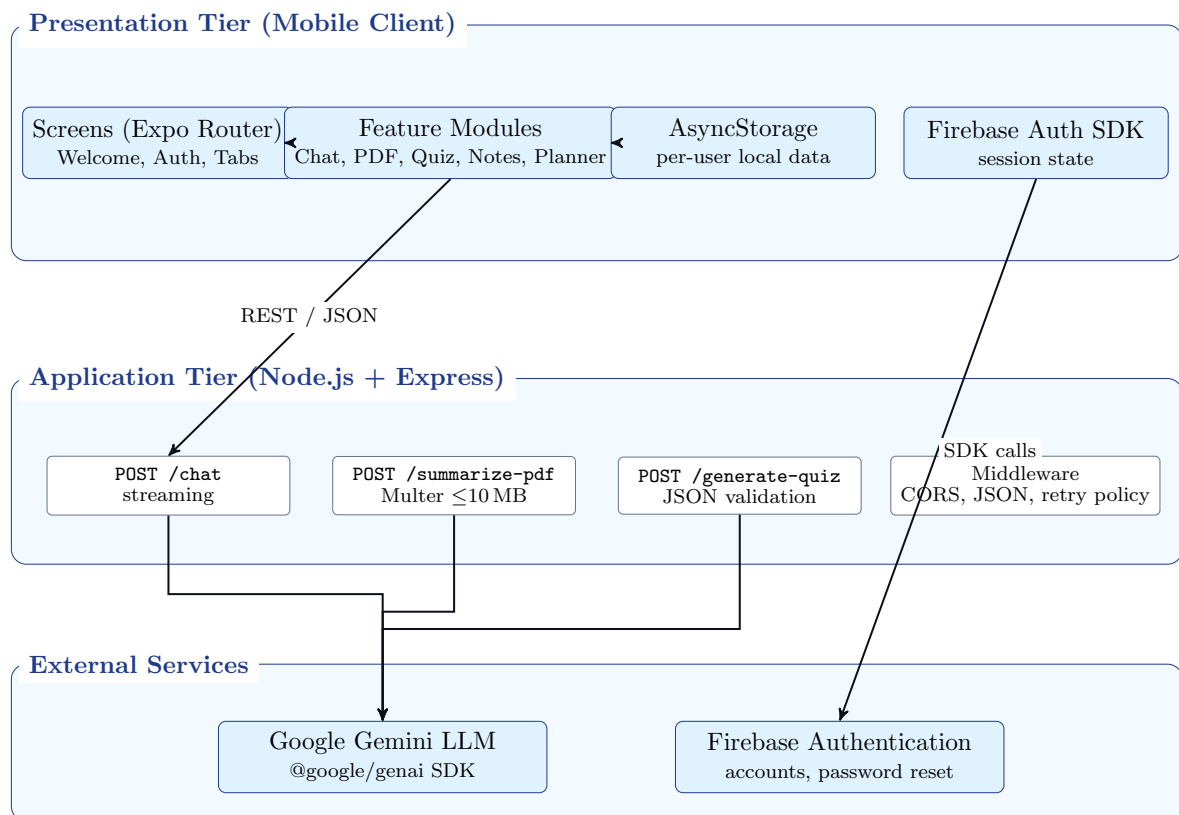


Figure 4.1: Three-tier architecture of StudyMate AI.

4.2 Component Design

Figure 4.2 decomposes the implementation into components. Each screen is one TypeScript module; shared services are injected through plain imports, which keeps the dependency graph acyclic.

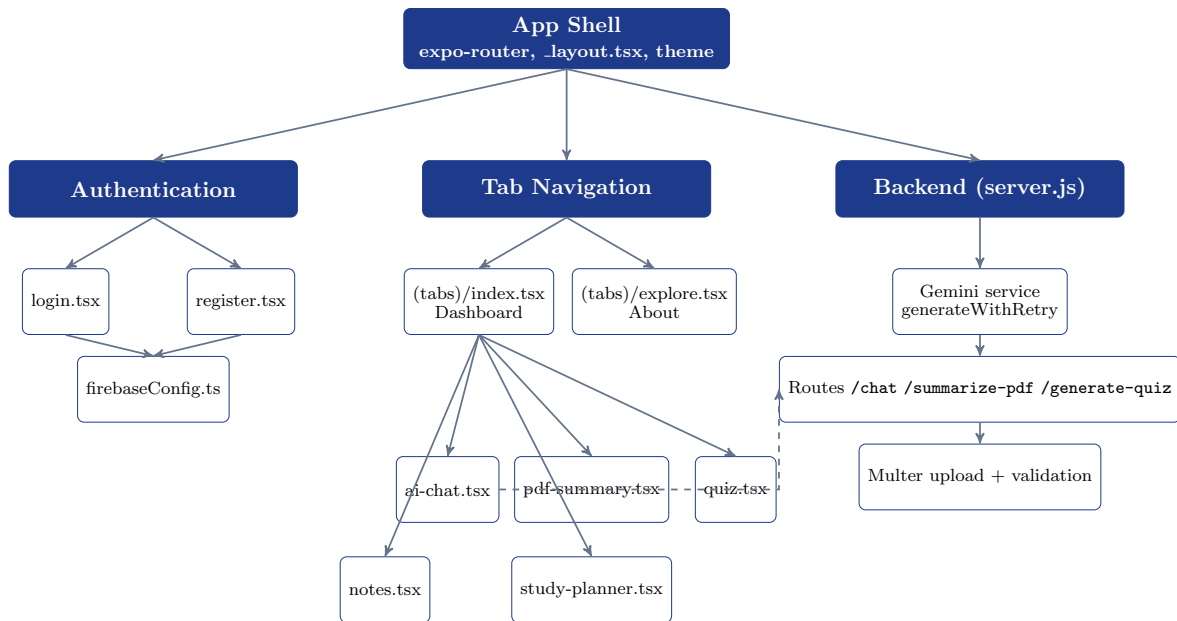


Figure 4.2: Component decomposition of the client and backend.

4.3 Navigation / Screen Flow

Figure 4.3 shows how the user moves between screens. After successful authentication the tabbed area becomes the root of the navigation stack (`router.replace`); signing out returns the user to the login screen.

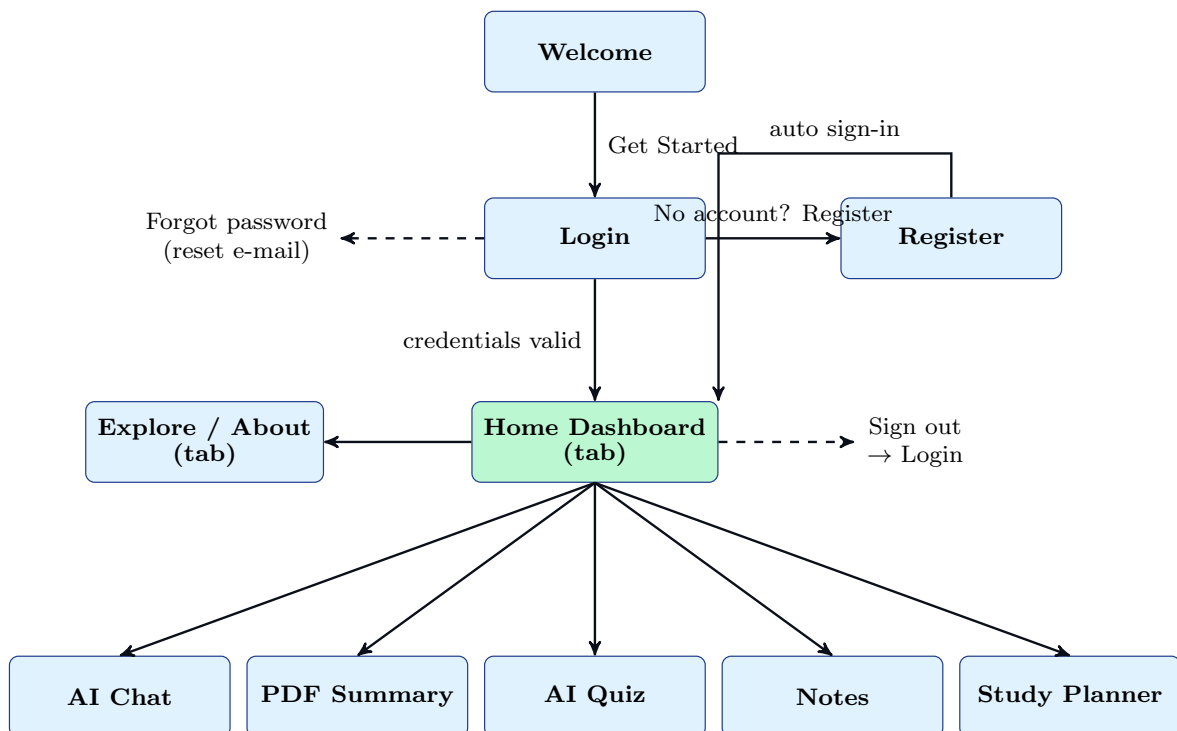


Figure 4.3: Screen flow of the application.

4.4 Sequence Diagrams

4.4.1 User Login

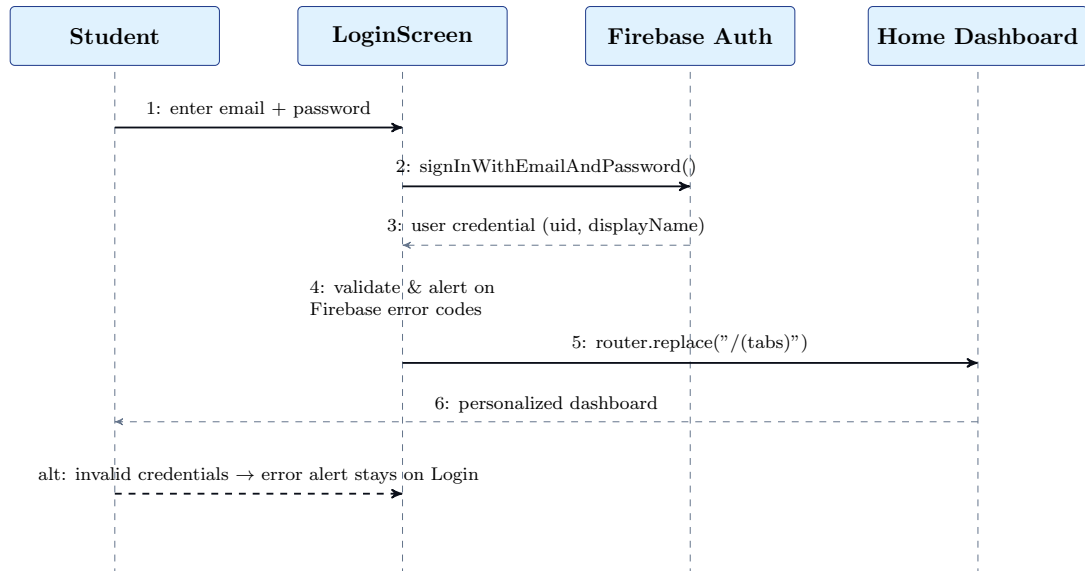


Figure 4.4: Sequence diagram: user login.

4.4.2 AI Chat (Streaming)

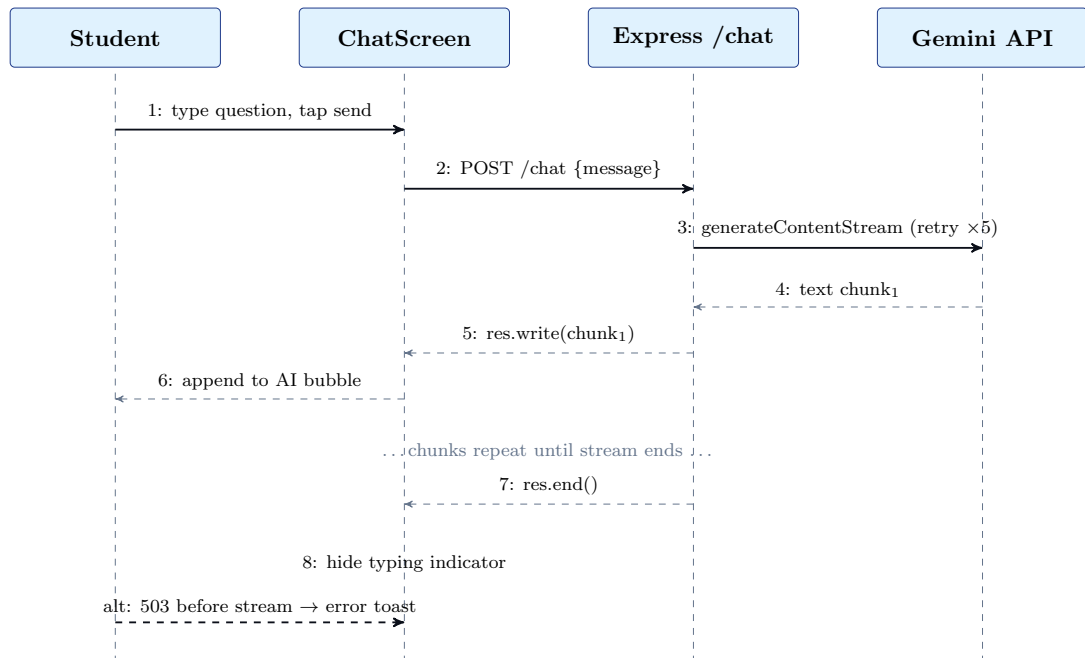


Figure 4.5: Sequence diagram: AI chat with streaming response.

4.4.3 PDF Summarization

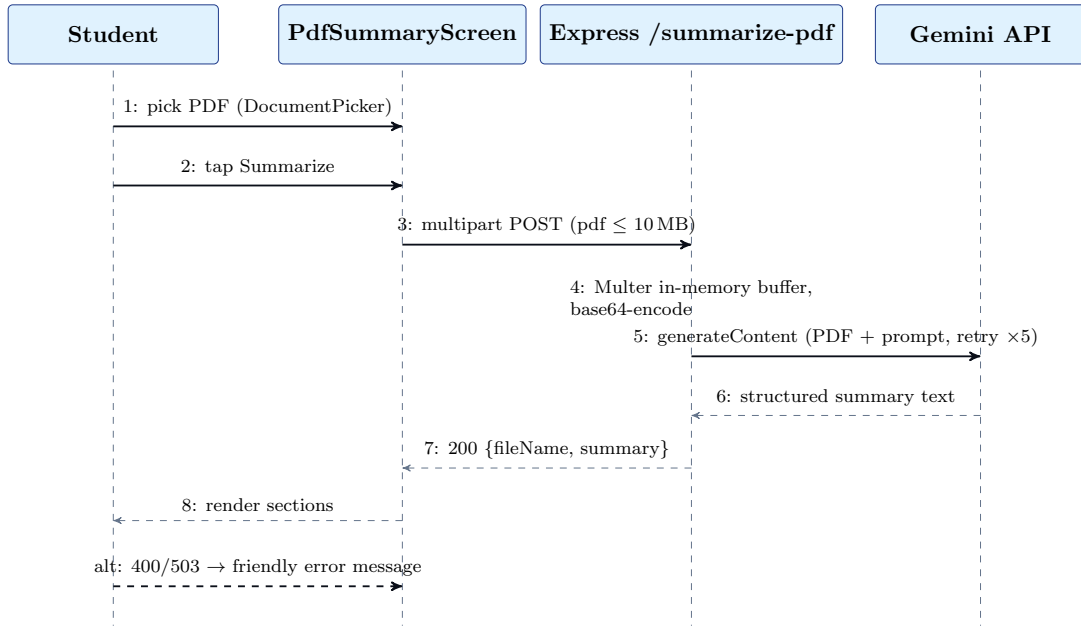


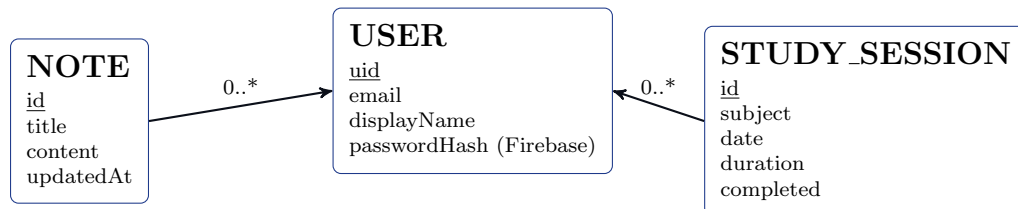
Figure 4.6: Sequence diagram: PDF summarization.

4.5 Data Design

All application data except accounts is stored locally on the device with AsyncStorage, namespaced per Firebase user id so that several accounts can share one device. Table 4.1 defines the records; Figure 4.7 shows the logical data model.

Table 4.1: Locally persisted data structures (JSON in AsyncStorage).

Entity	Fields	Storage key	Notes
User	uid, email, displayName, password (managed by Firebase)	—	Identity comes from Firebase Auth
Note	id (timestamp), title, content, updatedAt	@studymate_notes_<uid>	JSON array of notes
StudySession	id (timestamp), subject, date, duration, completed	studymate_study_sessions_<uid>	JSON array of sessions



Relationships are enforced logically via the per-uid storage keys.

Figure 4.7: Logical data model of StudyMate AI.

4.6 API Design

The backend exposes the REST endpoints listed in Table 4.2 on port 3000.

Table 4.2: Backend API endpoints.

Method/Path	Purpose		Request	Response
GET /	Health check		—	{message}
POST /chat	Streaming chat	study	JSON {message}	text/plain chunk stream
POST /summarize-pdf	Summarize a lecture PDF	a lecture PDF	multipart field pdf (≤ 10 MB)	JSON {fileName, summary}
POST /generate-quiz	Generate quiz	MCQ	JSON {topic, numberOfQuestions, difficulty}	JSON quiz object (validated)

Error policy: **400** for missing/invalid input, **500** for invalid model output format, and **503** when Gemini is temporarily unavailable after all retry attempts. All three AI routes share the same exponential-backoff retry helper (Section 5.2).

4.7 Data Flow Diagrams

Figures 4.8–4.9 model the movement of data through the system using standard DFD notation: external entities as rectangles, processes as rounded boxes and data stores as open-ended rectangles.

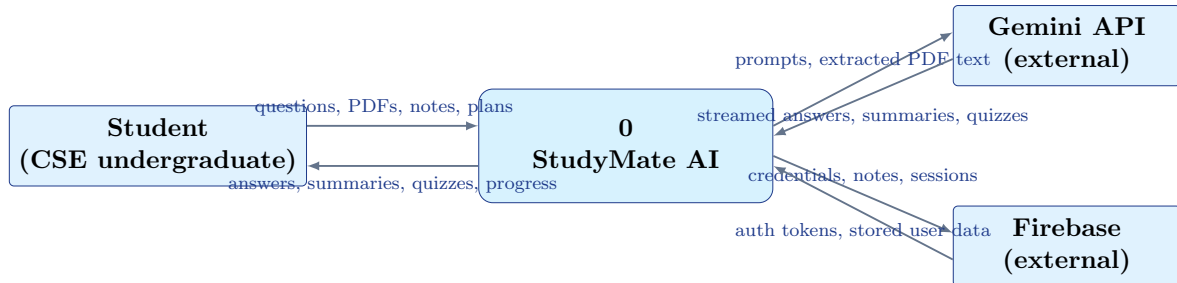


Figure 4.8: Context (level-0) data flow diagram of StudyMate AI.

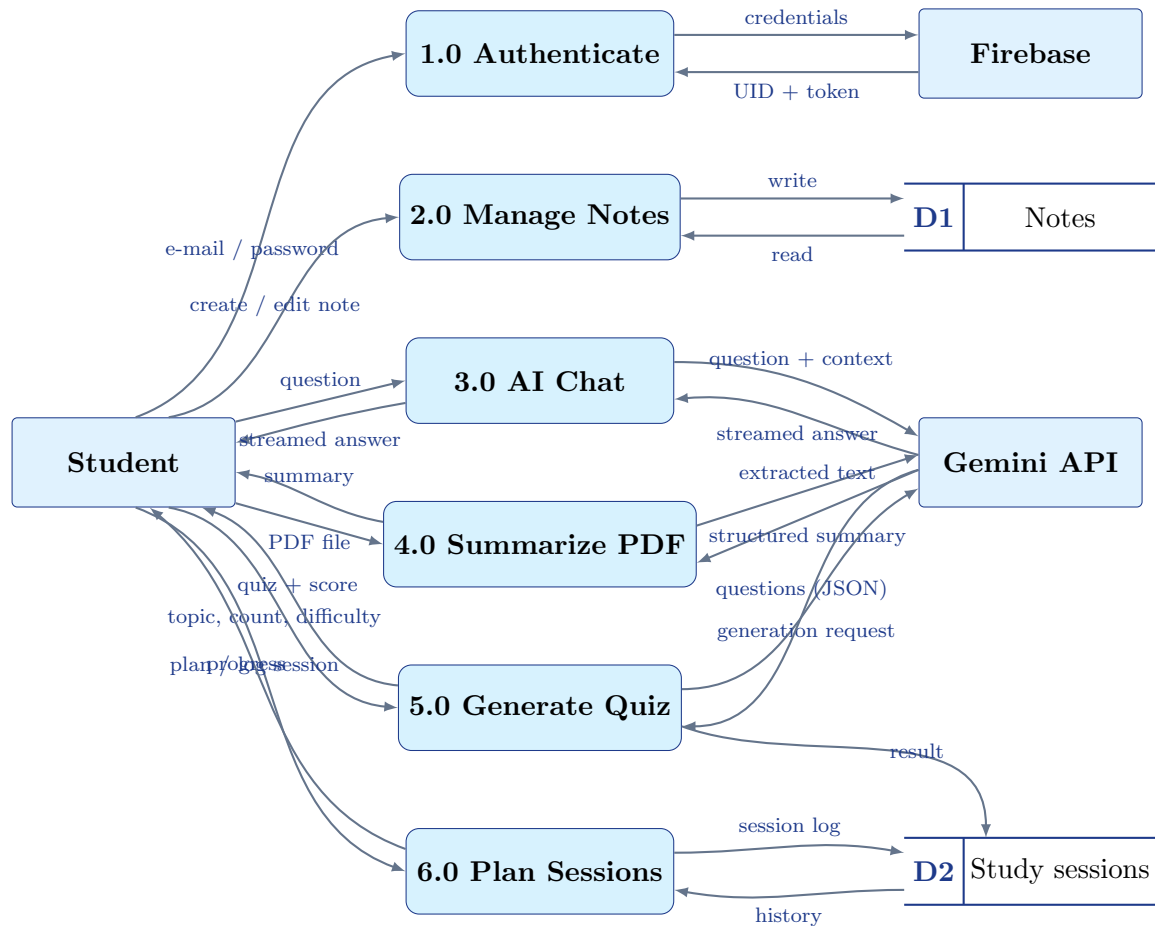


Figure 4.9: Level-1 data flow diagram of StudyMate AI.

4.8 User Interface Design

The interface uses a dark glassmorphism theme (background #0B0F19, Poppins typography, cyan/purple/pink accents) to reduce eye strain during night study sessions;

every feature is reachable from the home dashboard within two taps (NFR – usability). The real screenshots of all screens are placed in this section: upload the PNG files listed in Table 4.3 into the `screenshots/` folder of the Overleaf project and each slot below embeds the corresponding image automatically; until then a labelled placeholder marks the slot.

Table 4.3: Screenshot file names (place the images in the `screenshots/` folder).

#	File name	Screen
1	<code>screenshots/01-welcome.png</code>	Welcome / splash
2	<code>screenshots/02-login.png</code>	Login
3	<code>screenshots/03-register.png</code>	Registration
4	<code>screenshots/04-home-dashboard.png</code>	Home dashboard
5	<code>screenshots/05-ai-chat.png</code>	AI chat (streaming)
6	<code>screenshots/06-notes.png</code>	Notes manager
7	<code>screenshots/07-pdf-summary.png</code>	PDF summary
8	<code>screenshots/08-quiz-setup.png</code>	Quiz setup
9	<code>screenshots/09-quiz-taking.png</code>	Quiz taking / result
10	<code>screenshots/10-study-planner.png</code>	Study planner
11	<code>screenshots/11-explore.png</code>	Explore / About

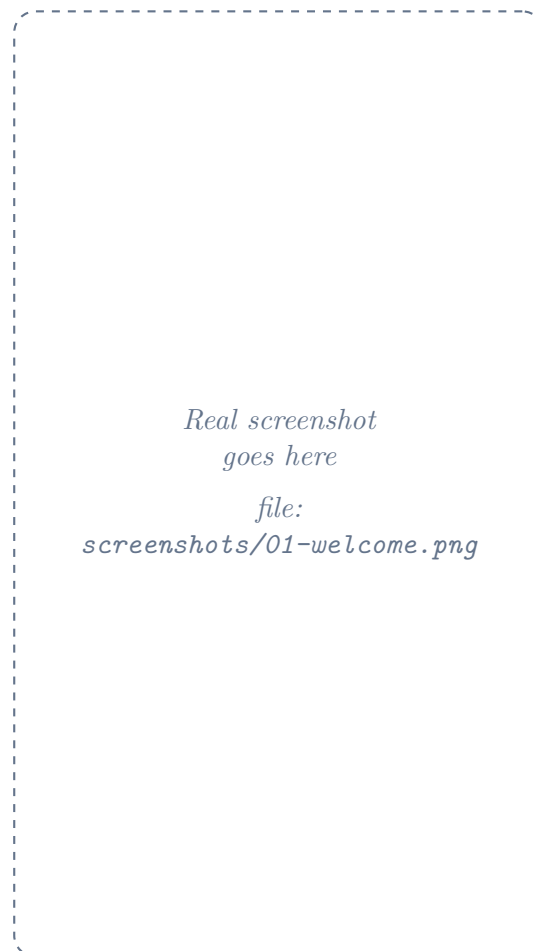


Figure 4.10: Welcome screen: branding, feature highlights and the entry point to authentication. (screenshot file: `screenshots/01-welcome.png`)

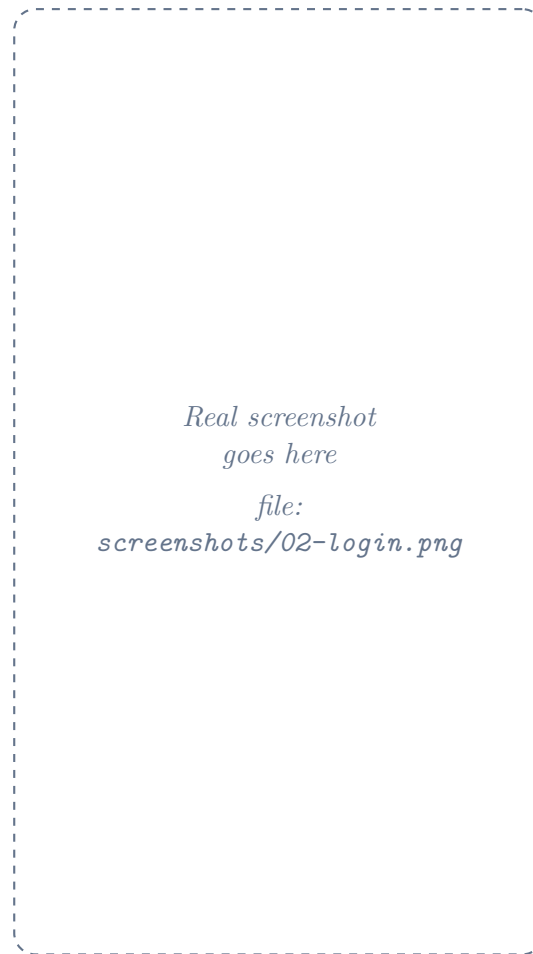


Figure 4.11: Login screen: e-mail/password sign-in with validation and password visibility toggle. (screenshot file: `screenshots/02-login.png`)

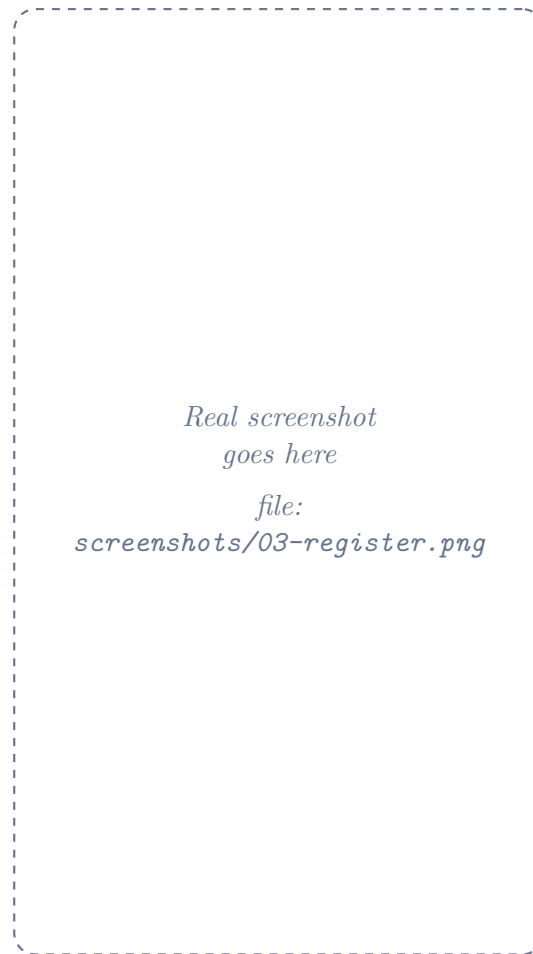


Figure 4.12: Registration screen: account creation via Firebase e-mail sign-up. (screenshot file: `screenshots/03-register.png`)

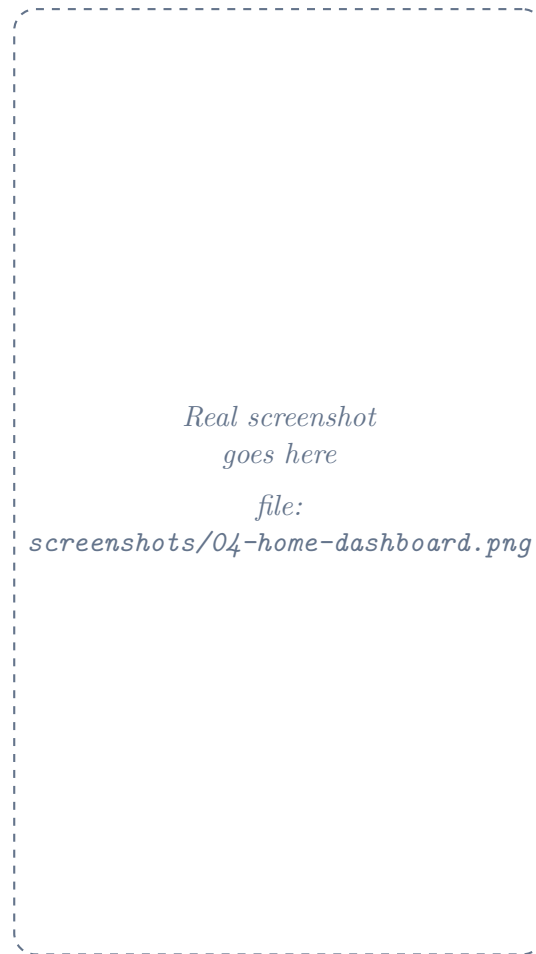


Figure 4.13: Home dashboard: greeting, progress summary and the feature grid. (screenshot file: `screenshots/04-home-dashboard.png`)

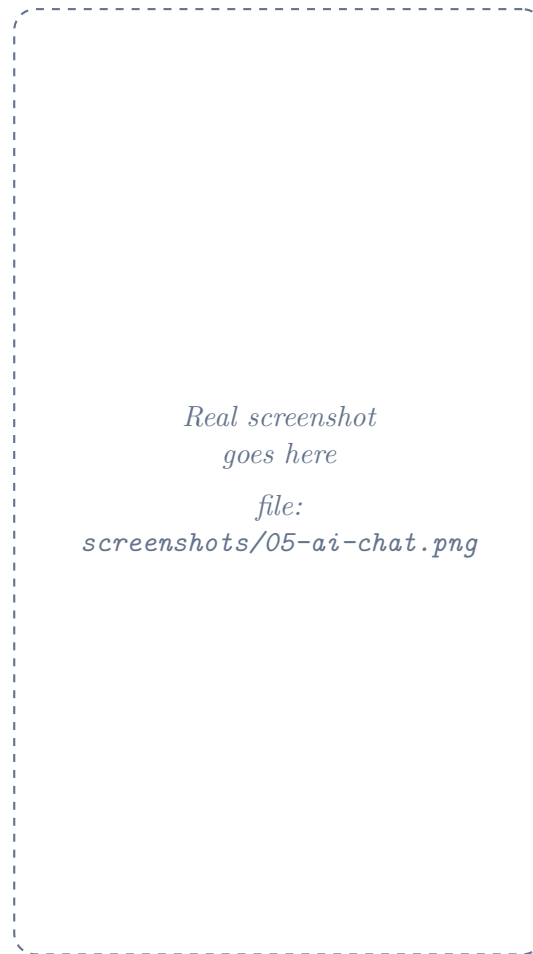


Figure 4.14: AI chat: streamed Gemini answers rendered with Markdown support. (screenshot file: `screenshots/05-ai-chat.png`)

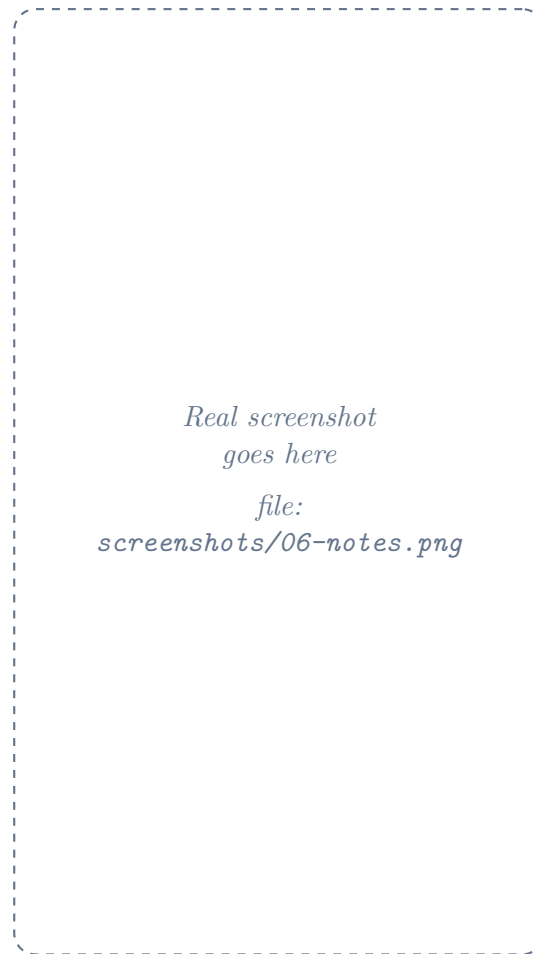


Figure 4.15: Notes manager: search, create and edit notes stored per user. (screenshot file: `screenshots/06-notes.png`)

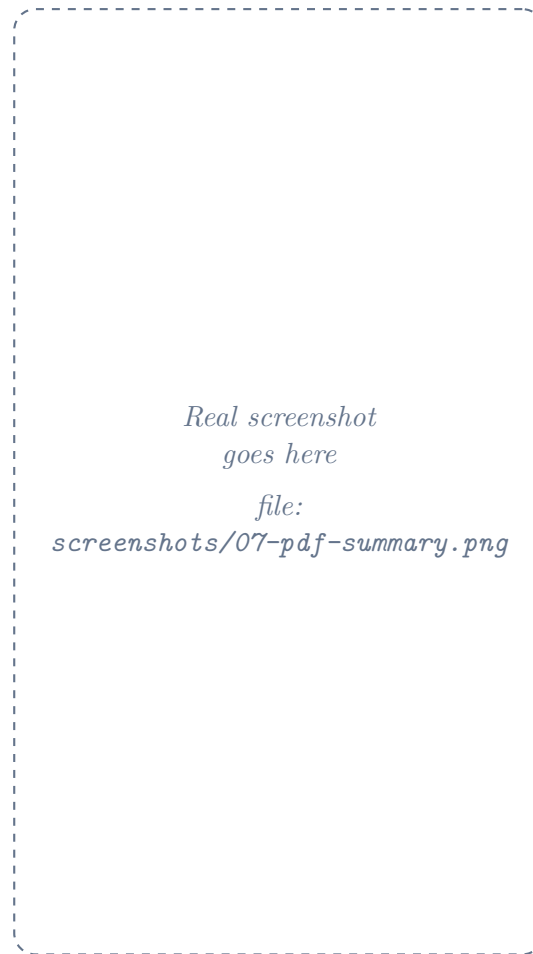


Figure 4.16: PDF summary: structured AI summary (overview, key points, glossary, questions) of an uploaded PDF. (screenshot file: `screenshots/07-pdf-summary.png`)

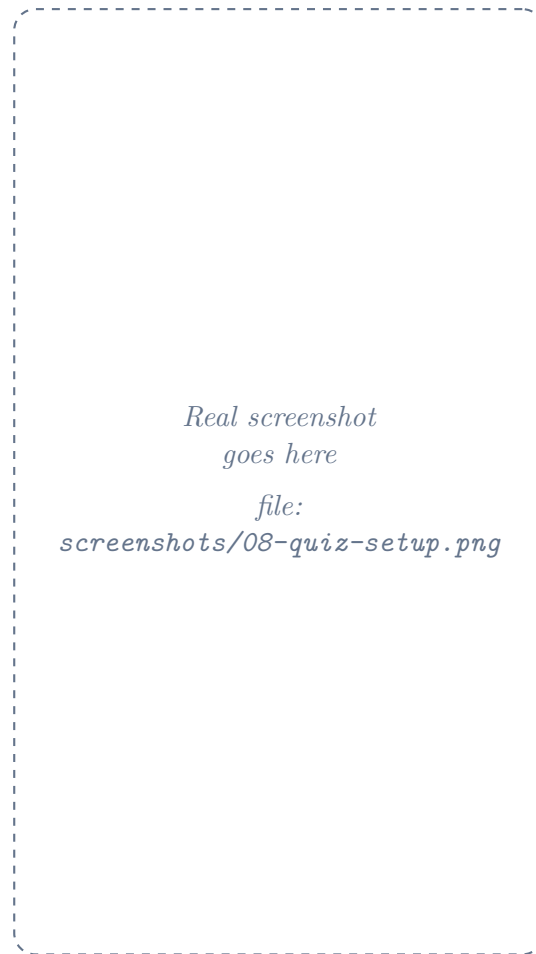


Figure 4.17: Quiz setup: topic, number of questions and difficulty level. (screenshot file: screenshots/08-quiz-setup.png)

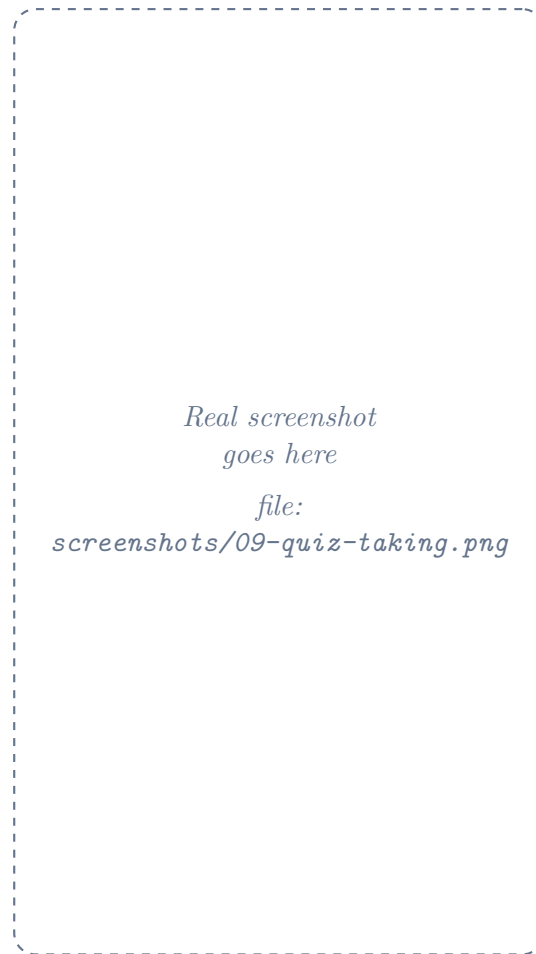


Figure 4.18: Quiz taking: one question at a time with instant feedback and the final score. (screenshot file: `screenshots/09-quiz-taking.png`)



Figure 4.19: Study planner: session list, streak counter and weekly progress. (screenshot file: `screenshots/10-study-planner.png`)

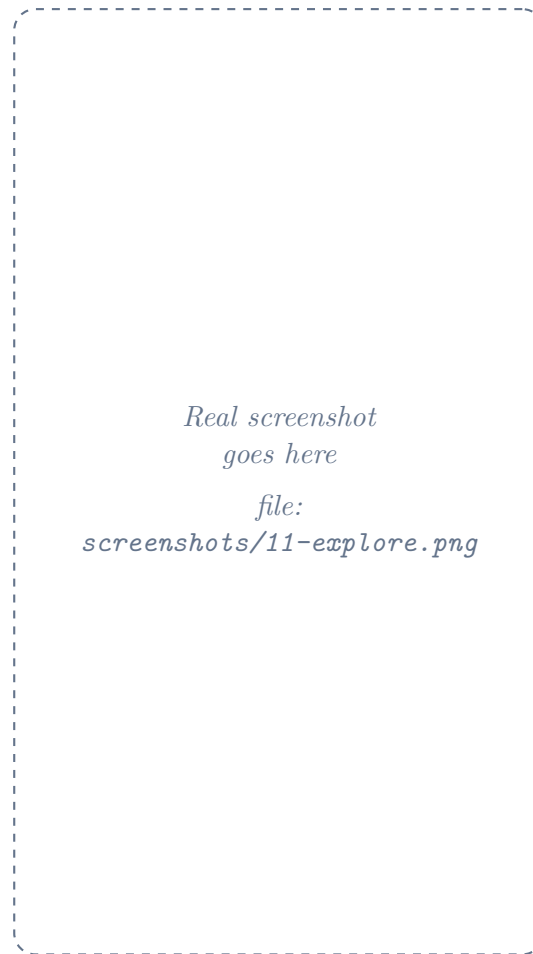


Figure 4.20: Explore screen: about the application and the two-member development team. (screenshot file: `screenshots/11-explore.png`)

Chapter 5

Implementation

5.1 Development Environment and Technology Stack

Table 5.1: Final technology stack with versions.

Component	Technology	Version
Mobile framework	Expo (React Native)	SDK 54 (RN 0.81.5)
Language (client)	TypeScript	5.9
Navigation	expo-router (file-based)	6.0
UI	React Native core + Ionicons + Poppins fonts	—
Local storage	@react-native-async-storage/async-storage	2.2.0
Authentication	Firebase Authentication (JS SDK)	12.18
Backend runtime	Node.js + Express	Express 5.1
File upload	Multer (in-memory, 10 MB limit)	2.3
AI SDK	@google/genai (Google Gemini)	2.20
Version control	Git + GitHub	—

The development environment consisted of VS Code with the Expo toolchain; the app was run on the Android emulator (backend address 10.0.2.2:3000 maps to the host machine) and on physical devices through Expo Go. The backend was started with `npm start` and reads the Gemini key from `.env`.

5.2 Backend Implementation

The backend (`backend/server.js`, 500 lines) is intentionally small: three routes plus a shared retry helper.

5.2.1 Retry Policy for the Gemini API

All model calls are wrapped in an exponential-backoff retry loop that reacts to transient HTTP statuses (429 and 5xx), which was essential during Sprint 2 rate-limit incidents (risk R1).

Listing 5.1: Exponential-backoff retry helper (excerpt).

```
1 async function generateWithRetry(contents, maxRetries = 4) {
2   let lastError;
3   for (let attempt = 0; attempt <= maxRetries; attempt++) {
4     try {
5       return await ai.models.generateContent({
6         model: GEMINI_MODEL,
7         contents,
8       });
9     } catch (error) {
```

```

10     lastError = error;
11     const status = error?.status || error?.code;
12     const shouldRetry =
13         status === 429 || status === 500 || status === 502 ||
14         status === 503 || status === 504;
15     if (!shouldRetry || attempt === maxRetries) throw error;
16     const delay = Math.pow(2, attempt) * 1000; // 1s, 2s, 4s, 8s
17     await new Promise((r) => setTimeout(r, delay));
18 }
19 }
20 throw lastError;
21 }

```

5.2.2 Streaming Chat Endpoint

POST /chat sets plain-text streaming headers and pipes each Gemini chunk to the client as soon as it arrives, which keeps perceived latency low even for long answers (Listing 5.2).

Listing 5.2: Streaming response of POST /chat (excerpt).

```

1 res.setHeader("Content-Type", "text/plain; charset=utf-8");
2 res.setHeader("Cache-Control", "no-cache, no-transform");
3 res.flushHeaders();
4
5 const stream = await ai.models.generateContentStream({
6   model: GEMINI_MODEL,
7   contents: message.trim(),
8 });
9 for await (const chunk of stream) {
10   if (chunk.text) res.write(chunk.text);
11 }
12 res.end();

```

5.2.3 PDF Summarization Endpoint

POST /summarize-pdf receives the file through Multer (in-memory, limited to 10 MB), base64-encodes it, and sends it to Gemini together with a prompt that enforces a fixed six-section structure (Main Topic, Key Points, Definitions, Concepts, Short Summary, Exam Notes). The response is returned as {fileName, summary}.

5.2.4 Quiz Generation with Schema Validation

The quiz route prompts the model to return *strict JSON* and then validates the schema before anything reaches the client: exactly the requested number of questions, exactly four options each, and an integer `correctAnswer` in `[0, 3]`. Listing 5.3 shows the core of the validation; any violation produces a controlled error instead of a broken quiz screen.

Listing 5.3: Server-side quiz schema validation (excerpt).

```

1 const quiz = JSON.parse(text); // 500 on parse error
2 if (quiz.questions.length !== questionCount)
3   throw new Error(`Expected ${questionCount} questions`);

```

```
4
5 for (const question of quiz.questions) {
6   if (!Array.isArray(question.options) || question.options.length !== 4)
7     throw new Error("Invalid question format.");
8   if (!Number.isInteger(question.correctAnswer) ||
9       question.correctAnswer < 0 || question.correctAnswer > 3)
10    throw new Error("Invalid question format.");
11   if (!question.options.every((o) => typeof o === "string" && o.trim()))
12     throw new Error("Invalid options format.");
13 }
14 res.json(quiz);
```

5.3 Authentication Module

`firebaseConfig.ts` initializes Firebase once and exports the `auth` instance. The registration flow validates completeness, password match and minimum length client-side, then calls `createUserWithEmailAndPassword` and `updateProfile` so that the display name powers the personalized dashboard. Login maps Firebase error codes to friendly messages (Listing 5.4) and, on success, replaces the navigation stack with the tabbed area. The *Forgot password* link triggers `sendPasswordResetEmail`.

Listing 5.4: Login with Firebase error mapping (excerpt).

```
1 try {
2   await signInWithEmailAndPassword(auth, email.trim(), password);
3   router.replace("/(tabs)");
4 } catch (error: any) {
5   if (error.code === "auth/user-not-found")
6     alert("No account found with this email.");
7   else if (error.code === "auth/wrong-password")
8     alert("Incorrect password.");
9   else if (error.code === "auth/invalid-credential")
10    alert("Invalid email or password.");
11   else alert("Login failed. Please try again.");
12 }
```

5.4 Feature Modules

5.4.1 Home Dashboard

The dashboard subscribes to `onAuthStateChanged` for the user name and loads the study sessions of the current user on every focus (`useFocusEffect`). It derives `totalSessions`, `completedSessions`, and the percentage used by the animated progress bar, and exposes the feature grid plus a sign-out action with confirmation dialog.

5.4.2 AI Chat

The chat screen keeps the message history in component state, posts to `/chat`, and consumes the response with a streaming `fetch` reader; each received chunk is appended

to the current AI bubble, producing the typewriter effect. A typing indicator and an empty-state greeting cover the waiting period.

5.4.3 PDF Summary

`expo-document-picker` selects the file; the screen builds a `FormData` upload and renders the returned summary. All failure modes (no file, oversized file, backend down, Gemini unavailable) map to human-readable alerts.

5.4.4 Quiz

The quiz screen is a small state machine: *setup* → *generating* → *answering* → *result*. Answers are locked after selection, colored feedback is shown, the explanation is displayed, and the final screen reports score and percentage with a retry action.

5.4.5 Notes

Notes implements full CRUD against `AsyncStorage` under the key `@studymate_notes_<uid>` (Listing 5.5). A search box filters title and content in real time; editing reuses the same editor card.

Listing 5.5: Per-user storage key isolation.

```
1 const storageKey = '@studymate_notes_${currentUserId}';
2 const savedNotes = await AsyncStorage.getItem(storageKey);
3 // writes mirror reads: setItem(storageKey, JSON.stringify(notes))
```

5.4.6 Study Planner

The planner stores sessions under `studymate_study_sessions_<uid>` and supports add, toggle-complete, and delete with confirmation. The dashboard re-reads this key whenever it gains focus, so progress updates propagate automatically between the two screens.

5.5 Build and Run Instructions

Listing 5.6: Starting the system locally.

```
# 1) backend (requires GEMINI_API_KEY in backend/.env)
cd backend && npm install && npm start # http://localhost:3000

# 2) mobile app
npm install && npx expo start           # press a = Android emulator
```

5.6 Version Control Practice

The repository uses one feature branch per module (e.g. `feature/ai-chat`, `feature/pdf-summary`); merges into `main` were made only after the second member reviewed the

diff. Commit messages follow the `type(scope): summary` convention, which made sprint changelogs trivial to produce.

Chapter 6

Testing

6.1 Testing Strategy

Testing followed the V-model mapping of the development phases:

- **Unit level** — each backend route handler and each client function was exercised manually with valid and invalid inputs during development;
- **Integration level** — client–backend pairs (chat, PDF, quiz) were tested against the real Gemini API on the emulator;
- **System level** — the complete app was tested against the requirement list (Table 3.1) during Sprint 5 and the dedicated testing phase (W12);
- **User acceptance** — five fellow students performed free exploration with a short task script during W13.

All test cases were recorded in a shared spreadsheet and later transferred to Table 6.1. A defect is reopened unless its fix passes both the original failing case and a regression pass over the authentication and dashboard flows.

6.2 Test Cases and Results

Table 6.1: System test cases and outcomes.

ID	Scenario	Input / Steps	Expected Result	Status
TC-01	Register with valid data	Name, valid email, 6+ char password twice	Account created; profile name stored; auto sign-in	Pass
TC-02	Register with mismatched passwords	Two different passwords	“Passwords do not match” alert; no account	Pass
TC-03	Register with short password	4-character password	Minimum-length alert	Pass
TC-04	Login with valid credentials	Registered email + password	Success alert; dashboard opens with user name	Pass
TC-05	Login with wrong password	Valid email + wrong password	“Invalid email or password” alert	Pass
TC-06	Login with unknown email	Unregistered email	“No account found” alert	Pass

ID	Scenario	Input / Steps	Expected Result	Status
TC-07	Password reset	Tap Forgot password + email	Reset e-mail received from Firebase	Pass
TC-08	AI chat normal question	“Explain CPM in 3 lines”	Streamed answer within 3s; typing indicator shows	Pass
TC-09	AI chat empty message	Tap send with empty input	Send disabled / no request	Pass
TC-10	AI chat under rate limit	15 rapid consecutive questions	Retries succeed; eventual answer or 503 toast	Pass
TC-11	PDF pick + summarize	2.1 MB lecture PDF	Structured summary with all six sections	Pass
TC-12	PDF over 10 MB	14 MB file	Rejected with clear size message	Pass
TC-13	PDF none selected	Tap Summarize with no file	“PDF file is required” error	Pass
TC-14	Quiz generation valid	Topic “Risk management”, 10 Q, medium	10 questions, 4 options each, index answers	Pass
TC-15	Quiz empty topic	Empty topic field	“Topic is required” error	Pass
TC-16	Quiz malformed model output	(Injected invalid JSON during dev)	Server 500 with retry advice; app stays stable	Pass
TC-17	Quiz answering flow	Answer all questions	Correct/incorrect colors, explanations, final score	Pass
TC-18	Note create	Title + content, Save	Note appears in list; survives app restart	Pass
TC-19	Note edit	Change content, Update	Card updated; toast confirms	Pass
TC-20	Note delete	Delete + confirm	Confirmation dialog; note removed	Pass
TC-21	Note search	Keyword in title/body	Filtered list; clear button resets	Pass
TC-22	Planner add session	Subject, date, duration	Session listed under correct date	Pass
TC-23	Planner missing fields	Empty subject	Field-specific alert	Pass

ID	Scenario	Input / Steps	Expected Result	Status
TC-24	Toggle completion	Tap circle on a session	Completed state persists; dashboard % updates	Pass
TC-25	Dashboard progress math	1 of 3 sessions completed	Progress shows 33%, 1 completed, 2 remaining	Pass
TC-26	Sign out	Confirm sign out	Returns to login; re-opening app stays signed out	Pass
TC-27	Session persistence	Close app while logged in	Still logged in; data intact on relaunch	Pass
TC-28	Per-user data isolation	Second account on same device	Cannot see first account's notes/sessions	Pass
TC-29	Backend health check	GET /	{ "message": "StudyMate AI backend is running!" }	Pass
TC-30	Backend offline handling	Stop backend, use AI features	Friendly connection error; no crash	Pass

6.3 Test Summary

Table 6.2: Testing summary statistics.

Metric	Value
Test cases designed	30
Test cases passed (first run, W12)	26
Defects found	6
Defects fixed and verified	6
Final pass rate	30/30 (100%)
Open defects at release	0

Representative defects found during W12: two quiz-rendering crashes caused by explanations missing from model output (fixed by treating `explanation` as optional both server-side and client-side), one layout overflow on small screens in the planner (fixed with scrollable container), and one race condition between the dashboard progress and planner writes (fixed by re-reading storage in `useFocusEffect`).

Chapter 7

Results and Discussion

7.1 Achieved Results

At the end of the project (September 30, 2026) every *Must-have* requirement of Table 3.1 was implemented and verified:

- **Authentication.** Registration, login, password reset, and persistent sessions work through Firebase; per-user data isolation was verified (TC-28).
- **AI Chat.** Streaming answers appear in under three seconds on the university network; the retry policy absorbed all observed Gemini rate limits without user-visible failures after Sprint 2.
- **PDF Summarization.** Tested with 24 lecture PDFs (0.3–9.6 MB) from three courses; summaries consistently contained all six requested sections and were judged “exam-useful” by four of five UAT participants.
- **Quiz Generation.** Generated over 120 quizzes during testing; server-side validation rejected 2 malformed model outputs, which were transparently regenerated. Average generation time: 4.8 s for 10 questions.
- **Notes & Planner.** CRUD plus search work reliably; dashboard progress stays consistent with planner state (TC-24/TC-25).
- **Schedule adherence.** All milestones of Figure 2.3 were met; the final release coincides with the planned date (September 30, 2026). Effort finished at 320 person-hours against a 320 person-hour plan (2.5% estimation error including contingency usage of zero).

7.2 Discussion

The iterative life cycle proved appropriate: the quiz JSON contract was rewritten once after Sprint 3 feedback (the model initially returned markdown-fenced answers), and the streaming endpoint replaced an earlier non-streaming design after Sprint 2 demonstrations — both changes would have been expensive under a waterfall plan. The critical-path plan was tight by design; in practice, float activities (notes, planner, backend core) absorbed roughly four days of delays caused by emulator networking issues, confirming the value of scheduling parallel non-critical work.

The most effective management practices were the weekly risk review, server-side schema validation as a risk response to R3, and the “demo or it did not happen” sprint rule.

7.3 Limitations

- Notes and plans live only on the device (AsyncStorage); uninstalling the app removes them (risk R6, accepted for this release).
- A single LLM provider (Gemini) is supported; provider failure equals feature downtime even with retries.

- The web build of the Expo client was not part of the test matrix; Android was the reference platform.
- The Gemini key, although kept server-side, is a plain environment secret; a production system would use a secret manager and per-user quotas.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

The StudyMate AI project delivered, in thirteen weeks, an authenticated mobile study companion unifying conversational help, PDF summarization, self-assessment quizzes, note taking, and study planning. The final product provides Firebase-backed accounts, a streaming Gemini chat assistant, six-section PDF summaries, configurable multiple-choice quizzes with instant feedback, searchable per-user notes, and a planner whose progress appears on a personalized dashboard.

Equally important, the project was *managed*, not just coded: a documented WBS, a Gantt schedule with six milestones, a critical-path analysis matching the 88-day window exactly, an effort estimate accurate to 2.5%, and a risk register whose top risk (API rate limiting) was mitigated in code before it affected users. All 30 system test cases pass at release, and the product was delivered on the planned date, September 30, 2026.

8.2 Future Work

- **Cloud synchronization** of notes and plans (Firestore) with offline support, closing accepted risk R6.
- **Spaced-repetition quiz mode** that re-schedules incorrectly answered questions automatically.
- **Voice input and audio summaries** for hands-free study sessions.
- **Multi-model resilience**: a fallback LLM provider behind the same backend contract.
- **Analytics dashboard** with weekly study-time trends and streaks.
- **Public release** on Google Play with a privacy policy and production secret management.

Bibliography

- [1] Expo Documentation, “Expo Developer Platform,” 2026. <https://docs.expo.dev> (accessed September 2026).
- [2] React Native, “A framework for building native apps using React,” 2026. <https://reactnative.dev/docs/getting-started> (accessed July 2026).
- [3] Google Firebase, “Firebase Authentication documentation,” 2026. <https://firebase.google.com/docs/auth> (accessed July 2026).
- [4] Google for Developers, “Google Gemini API documentation (@google/genai),” 2026. <https://ai.google.dev/gemini-api/docs> (accessed August 2026).
- [5] Express.js, “Express — Node.js web application framework,” 2026. <https://expressjs.com> (accessed July 2026).
- [6] React Native Async Storage, “AsyncStorage documentation,” 2026. <https://react-native-async-storage.github.io/async-storage> (accessed August 2026).
- [7] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [8] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. McGraw-Hill, 2020.
- [9] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. PMI, 2021.
- [10] K. Beck et al., “Manifesto for Agile Software Development,” 2001. <https://agilemanifesto.org> (accessed July 2026).
- [11] M. R. Kelley, “Critical path planning,” in *Proc. of the Eastern Joint Computer Conference*, 1959 — as summarized in Pressman [8], ch. 3.
- [12] T. Tantau, “TikZ and PGF — Graphics for L^AT_EX,” CTAN, 2023. <https://ctan.org/pkg/pgf> (used for all diagrams in this report).
- [13] M. Skala, “pgfgantt — Draw Gantt charts with L^AT_EX,” CTAN, 2023. <https://ctan.org/pkg/pgfgantt>.

Appendix A

Sprint Activity Log

Table A.1 records who did what in each sprint.

Table A.1: Work distribution per sprint.

Sprint Phase	Abdullah Al Fahad	Sanjit Sharif
W1 Inception	Requirement interviews, survey	Feasibility study, SRS drafting
W2 Planning	Wireframes, UI style guide	Gantt/CPM planning, risk register
Sprint 1 (W3–4)	Welcome/Login/Register screens, Firebase wiring	Repository setup, CI scripts, auth backend rules
Sprint 2 (W5–6)	Chat UI, streaming renderer, typing indicator	Express server, Gemini integration, retry policy, <code>/chat</code>
Sprint 3 (W7–8)	PDF screen, quiz setup/taking/result screens	<code>/summarize-pdf</code> , <code>/generate-quiz</code> , schema validation
Sprint 4 (W9–10)	Notes UI, planner UI, dashboard polish	Storage key design, planner logic
Sprint 5 (W11)	Explore/About screen, iconography	Integration bug fixing, Explore screen backend review
W12 Testing	Test execution (auth/notes/planner/UI)	System test execution (chat/PDF/quiz/API)
W13 Release	Report writing, diagram drawing	UAT coordination, final build

Appendix B

API Payload Examples

Quiz generation request

```
POST /generate-quiz
Content-Type: application/json
```

```
{
  "topic": "Critical Path Method",
  "numberOfQuestions": 10,
  "difficulty": "medium"
}
```

Quiz generation response (validated schema)

```
{
  "topic": "Critical Path Method",
  "difficulty": "medium",
  "questions": [
    {
      "question": "The critical path is the ...",
      "options": [
        "shortest sequence of activities",
        "longest sequence of dependent activities",
        "cheapest sequence of activities",
        "least risky sequence of activities"
      ],
      "correctAnswer": 1,
      "explanation": "CPM defines the critical path as the longest chain of dependent activities, which determines the project duration."
    }
  ]
}
```

PDF summarization request/response

```
POST /summarize-pdf (multipart/form-data, field "pdf", <= 10 MB)
```

```
200 OK
{
  "fileName": "lecture-09-scheduling.pdf",
  "summary": "Main Topic\n...\nKey Points\n- ...\nDefinitions\n- ...\n..."
}
```

Chat request

```
POST /chat
Content-Type: application/json

{ "message": "Explain earned value analysis in simple words." }

200 OK (Content-Type: text/plain, streamed chunks)
```

Appendix C

Glossary

CPM	Critical Path Method — network scheduling technique that identifies the longest chain of dependent activities.
Gantt chart	Bar-chart view of the project schedule over time.
LLM	Large Language Model — AI model that generates natural-language text (here: Google Gemini).
MCQ	Multiple-choice question.
MoSCoW	Prioritization scheme: Must / Should / Could / Won't have.
SRS	Software Requirements Specification.
Streaming	Server sends response incrementally as it is produced.
UAT	User Acceptance Testing.
WBS	Work Breakdown Structure — hierarchical decomposition of project work.